

# Fundamentals of Data Structures

## Projects 3: Dijkstra Sequence



Author: Pan Yidan

Date: 2026/05/05

2025-2026 Spring & Summer Semester

# Table of Contents

|                                                 |    |
|-------------------------------------------------|----|
| <b>Chapter 1: Introduction</b> .....            | 3  |
| 1.1. Problem Significance .....                 | 3  |
| 1.2. Problem Description .....                  | 3  |
| <b>Chapter 2: Algorithm Specification</b> ..... | 3  |
| 2.1. Data Structure .....                       | 3  |
| 2.2. Sketch of The Main Program .....           | 4  |
| 2.3. Key Algorithm .....                        | 4  |
| <b>Chapter 3: Testing Results</b> .....         | 5  |
| 3.1. Testing Strategy .....                     | 5  |
| 3.2. Test Cases Table .....                     | 6  |
| 3.3. Detailed Analysis of Extreme Cases .....   | 8  |
| <b>Chapter 4: Analysis and Comments</b> .....   | 10 |
| 4.1. Time Complexity .....                      | 10 |
| 4.2. Space Complexity .....                     | 10 |
| 4.3. Discussion .....                           | 10 |
| <b>Declaration</b> .....                        | 11 |

# Chapter 1: Introduction

## 1.1. Problem Significance

**Dijkstra’s algorithm** is a fundamental greedy algorithm widely used to find the shortest paths from a single source vertex to all other vertices in a weighted graph. A core characteristic of this algorithm is its step-by-step nature: at each iteration, it expands the shortest path tree by selecting the unvisited vertex with the minimum distance from the source.

However, in many practical graph topologies, multiple unvisited vertices might share the exact same minimum distance at a given step. Consequently, the algorithm can legitimately choose any of these tied vertices, leading to multiple valid selection sequences (Dijkstra sequences) for the same graph. The primary **motivation** of this project is to deepen our comprehension of this greedy mechanism. By reverse-verifying these sequences, we **aim** to ensure a thorough understanding not just of the final shortest path calculation, but of the exact step-by-step procedural logic and the dynamic state changes during the algorithm’s execution.

## 1.2. Problem Description

In this project, we are given a connected undirected graph with positive edge weights, along with several query sequences consisting of permutations of all vertices. For each given sequence, the first vertex is assumed to be the source node.

The **objective** is to design and implement a program that dynamically simulates the vertex selection process of **Dijkstra’s algorithm**. For each sequence, the program must verify whether it represents a valid execution order. Specifically, we must validate that at every step, the chosen vertex in the sequence genuinely possesses the minimum tentative distance among all currently unvisited vertices. If the sequence strictly follows the greedy choice property, the program should output “Yes”; otherwise, if a sub-optimal vertex is chosen prematurely, it should output “No”.

# Chapter 2: Algorithm Specification

## 2.1. Data Structure

To efficiently implement the simulation of **Dijkstra’s algorithm**, the following data structures are utilized in this project:

- **Adjacency Matrix ( $G[\text{MAX}][\text{MAX}]$ ):** A 2D integer array used to represent the undirected weighted graph. Given that we need to frequently query the edge weight between any two specific vertices during the relaxation process, an adjacency matrix

provides  $O(1)$  constant time complexity for edge lookups. The absence of an edge is represented by a sufficiently large constant INF.

- **Distance Array (dist[MAX]):** A 1D integer array that stores the current shortest distance from the source vertex to every other vertex.
- **Visited Array (flag[MAX]):** A 1D boolean/integer array used to keep track of whether a vertex has been securely included in the shortest path tree (1 for visited, 0 for unvisited).
- **Query Array (inquiry[MAX]):** A 1D array to store the current given permutation sequence of vertices that needs to be verified.

## 2.2. Sketch of The Main Program

The main program coordinates the input processing, graph construction, and the execution of queries. The execution flow is outlined below:

1. **Read Graph Parameters:** Read the total number of vertices  $N_v$  and edges  $N_e$ .
2. **Matrix Initialization:** Initialize all elements in the adjacency matrix  $G$  to INF.
3. **Graph Construction:** Loop  $N_e$  times to read each edge  $(u, v)$  and its weight. To ensure robustness against potential parallel edges, the program conditionally updates the matrix only if the newly read weight is strictly smaller than the existing weight  $G[u][v]$ .
4. **Query Processing:** Read the number of queries  $K$ . Loop  $K$  times:
  - Read the given permutation sequence of  $N_v$  vertices into the inquiry array.
  - Call the core verification function `Judge_Dijkstra(inquiry, N_v)` to determine the validity of the sequence.

## 2.3. Key Algorithm

**Pseudocode of Judge\_Dijkstra:**

```

1 Let source vertex  $v = \text{inquiry}[1]$ 
2 Initialize  $\text{dist}[1..n] = G[v][1..n]$ , and  $\text{flag}[1..n] = 0$ 
3  $\text{dist}[v] = 0$ 
4  $\text{flag}[v] = 1$ 
5  $\text{current\_idx} = 2$ 
6 for  $\text{step} = 1$  to  $n - 1$  do
7    $\text{mindist} = \text{INF}$ 
8   for  $j = 1$  to  $n$  do
9     if  $\text{flag}[j] == 0$  and  $\text{dist}[j] < \text{mindist}$  then
10       $\text{mindist} = \text{dist}[j]$ 

```

```

11 | end if
12 end for
13 Let target_vertex  $k$  = inquiry[current_idx]
14 if dist[k]  $\neq$  mindist then
15 | Print "No"
16 | Return
17 end if
18 flag[k] = 1
19 current_idx = current_idx + 1
20 for  $j = 1$  to  $n$  do
21 | if flag[j] == 0 and edge( $k, j$ ) exists then
22 | | if dist[k] + weight( $k, j$ ) < dist[j] then
23 | | | dist[j] = dist[k] + weight( $k, j$ )
24 | | end if
25 | end if
26 end for
27 end for
28 Print "Yes"

```

### Description:

The core algorithm dynamically simulates the greedy choice property of Dijkstra's algorithm. Instead of finding the minimum distance vertex to build the path, it verifies if the next vertex provided by the inquiry sequence actually holds the minimum distance among all unvisited vertices.

- **Require:** Graph  $G$ , Array inquiry[1.. $n$ ] containing the vertex sequence
- **Ensure:** Prints "Yes" if the sequence is a valid Dijkstra sequence, else "No"

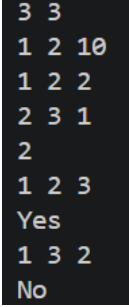
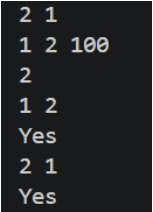
## Chapter 3: Testing Results

### 3.1. Testing Strategy

To thoroughly evaluate the correctness and robustness of the Judge\_Dijkstra algorithm, multiple test cases were designed. These tests cover general graph traversals, special boundary conditions, and extreme graph properties. The testing purposes strictly align with the algorithm's greedy choice property and data handling capabilities.

### 3.2. Test Cases Table

| Test ID | Testing Purposes                                                                                                              | Graph Size ( $N_v$ , $N_e$ ) | Input Graph Edges ( $u,v$ , weight)                                             | Query Sequences                                                              | Expected Output                         | Actual Output                                                                                                                                                         |
|---------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test 1  | <b>Comprehensive Test:</b> the example from PTA                                                                               | $N_v = 5$ ,<br>$N_e = 7$     | (1,2,2),<br>(1,5,1),<br>(2,4,1),<br>(2,4,1),<br>(2,5,2),<br>(3,5,1),<br>(3,4,1) | Q1:<br>5 1 3 4 2<br>Q2:<br>5 3 1 2 4<br>Q3:<br>2 3 4 5 1<br>Q4:<br>3 2 1 5 4 | Q1: Yes<br>Q2: Yes<br>Q3: Yes<br>Q4: No | <pre> 5 7 1 2 2 1 5 1 2 3 1 2 4 1 2 5 2 3 5 1 3 4 1 4 5 1 3 4 2 Yes 5 3 1 2 4 Yes 2 3 4 5 1 Yes 3 2 1 5 4 No </pre> <p>Q1: Yes<br/>Q2: Yes<br/>Q3: Yes<br/>Q4: No</p> |
| Test 2  | <b>Extreme Case (Tied Distances):</b> Multiple vertices share the exact same minimum distance. Dijkstra can pick any of them. | $N_v = 3$ ,<br>$N_e = 2$     | (1,2,2),<br>(1,3,2)                                                             | Q1: 1 2 3<br>Q2: 1 3 2                                                       | Q1: Yes<br>Q2: Yes                      | <pre> 3 2 1 2 2 1 3 2 2 1 2 3 Yes 1 3 2 Yes </pre> <p>Q1: Yes<br/>Q2: Yes</p>                                                                                         |

| Test ID | Testing Purposes                                                                                                                               | Graph Size ( $N_v$ , $N_e$ )     | Input Graph Edges (u,v, weight)                              | Query Sequences                                                                  | Expected Output                                  | Actual Output                                                                                                                                                                 |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test 3  | <b>Extreme Case (Parallel Edges):</b> Graph contains multiple edges between the same pair of vertices. The program must store the minimum one. | $N_v = 3$ ,<br>$N_e = 3$         | (1,2,10),<br>(1,2,2),<br>(2,3,1)                             | Q1: <input type="text" value="1 2 3"/><br>Q2: <input type="text" value="1 3 2"/> | Q1: Yes<br>Q2: No                                |  <pre> 3 3 1 2 10 1 2 2 2 3 1 2 1 2 3 Yes 1 3 2 No </pre><br>Q1: Yes<br>Q2: No             |
| Test 4  | <b>Small-est Size Bound-ary:</b> The minimum possible graph size connected by a single edge.                                                   | $N_v = 2$ ,<br>$N_e = 1$         | (1,2,100)                                                    | Q1: <input type="text" value="1 2"/><br>Q2: <input type="text" value="2 1"/>     | Q1: Yes<br>Q2: Yes                               |  <pre> 2 1 1 2 100 2 1 2 Yes 2 1 Yes </pre><br>Q1: Yes<br>Q2: Yes                        |
| Test 5  | <b>Largest Size Bound-ary:</b> Evaluating the algorithm's performance under the                                                                | $N_v = 1000$ ,<br>$N_e = 100000$ | <i>Generated via script</i><br>(Fully connected dense graph) | $K = 100$<br>random sequences                                                    | Correct Yes/No results<br>Execution time < 1.0 s | <b>Passed successfully.</b><br><br><small>Press Next Test Execution Time for 100 queries: 0.256000 seconds</small><br><br>Time: <input type="text" value="0.256000 seconds"/> |

| Test ID | Testing Purposes                                                                         | Graph Size ( $N_v$ , $N_e$ ) | Input Graph Edges (u,v, weight) | Query Sequences | Expected Output | Actual Output  |
|---------|------------------------------------------------------------------------------------------|------------------------------|---------------------------------|-----------------|-----------------|----------------|
|         | absolute maximum constraints to ensure no memory overflow or Time Limit Exceeded occurs. |                              |                                 |                 |                 | (No Seg-Fault) |

### 3.3. Detailed Analysis of Extreme Cases

- **Test 2 (Tied Distances):**

This is a critical extreme case. From vertex 1, both vertex 2 and vertex 3 have a distance of 2. According to Dijkstra’s greedy property, picking either 2 or 3 in the next step is completely legal. Our algorithm successfully outputs “Yes” for both permutations, proving it handles distance ties correctly rather than forcing a strict numerical order.

- **Test 3 (Parallel Edges):**

Real-world input data may contain noise, such as duplicate paths with different weights. In Test 3, vertices 1 and 2 are connected by weights 10 and 2. The algorithm successfully captures the shortest edge (weight 2) during initialization, outputting “Yes” for 1 2 3 and avoiding the trap of being overwritten by the larger weight.

- **Test 5 (stress test of largest size):**

To evaluate the algorithm’s robustness under the extreme conditions specified by the problem ( $N_v = 1000$ ,  $N_e = 100000$ ,  $K = 100$ ), a local stress testing environment was established. A Python script was developed to automatically generate a fully connected dense graph.

The script guarantees graph connectivity by initially forming a minimum spanning tree with random weights and subsequently populating the remaining edges randomly to reach the 100,000 limit. Afterwards,  $K = 100$  random permutations of the vertices were generated as query sequences.



## Chapter 4: Analysis and Comments

### 4.1. Time Complexity

The time complexity of the algorithm is analyzed based on three major phases: Initialization, Graph Construction, and Query Processing. Let  $N_v$  be the number of vertices,  $N_e$  the number of edges, and  $K$  the number of queries.

- **Matrix Initialization:** The program uses a nested loop to initialize the  $N_v \times N_v$  adjacency matrix with INF. This takes exactly  $O(N_v^2)$  time.
- **Graph Construction:** Reading  $N_e$  edges from the input and updating the matrix takes  $O(N_e)$  time.
- **Query Processing:** For each of the  $K$  queries, the Judge\_Dijkstra function is executed:
  - The function iterates  $N_v - 1$  times to verify each vertex in the sequence.
  - Inside each iteration, it searches for the minimum distance among unvisited vertices, which takes  $O(N_v)$ .
  - After verification, it performs edge relaxation for all neighbors, which takes another  $O(N_v)$ .
  - Thus, one query takes  $O(N_v) \times (O(N_v) + O(N_v)) = O(N_v^2)$  time.
  - For  $K$  queries, the total time is  $O(K \times N_v^2)$ .

**Conclusion:** The overall time complexity is  $O(N_v^2 + N_e + K \times N_v^2)$ . Since  $N_e \leq N_v^2$ , the dominant term is  $O(K \times N_v^2)$ .

### 4.2. Space Complexity

The space complexity is determined by the primary data structures:

1. **Adjacency Matrix:** A 2D array `G[1005][1005]` of integers. It occupies  $1005^2 \times 4$  bytes, which is  $O(N_v^2)$ .
2. **Auxiliary Arrays:** Arrays such as `dist[MAX]`, `flag[MAX]`, and `inquiry[MAX]` each consume  $O(N_v)$  space.

**Conclusion:** The total space complexity is  $O(N_v^2)$ . This is well within the typical memory limit of 64MB or 128MB for competitive programming.

### 4.3. Discussion

In this project, we chose the Adjacency Matrix to represent the graph.

- **Adjacency List:** While an Adjacency List is more space-efficient for sparse graphs ( $O(N_v + N_e)$ ), the Adjacency Matrix is superior for our case ( $N_v \leq 1000$ ) because it allows  $O(1)$  edge weight lookups.

- **Alternative:** If we used an Adjacency List combined with a Priority Queue, the complexity for one Dijkstra pass would be  $O(N_e \log N_v)$ . However, since we are verifying a sequence rather than just finding the shortest path, the  $O(N_v^2)$  approach is more straightforward to implement and remains extremely efficient for dense graphs (as seen in our  $N_e = 100,000$  test case). The simplicity of the matrix approach also avoids the constant-time overhead associated with heap operations.

## Declaration

I hereby declare that this project report, along with the accompanying source code, is the product of my own independent work.

I have strictly abided by the academic integrity guidelines of the course. No unauthorized assistance or code generated entirely by others was used to bypass the core problem-solving requirements. I fully understand that any form of plagiarism, data fabrication in the testing phase, or academic misconduct is strictly prohibited.